



Pokerbots 2026

Lecture 9: Counterfactual Regret Minimization (CFR)

Sponsors



CITADEL | CITADEL Securities





Announcements

HACKATHON PART 2

- Students expressed interest at pkr.bot/hack
- Thursday, Jan 22nd, 7pm — LATE (room TBD)
 - However late it ends up going
- Awesome food/drinks
- *More prizes/games*
- See you there!



Wednesday 1/28

Guest Lecture: Dr. Noam Brown

- Won Pokerbots and co-created Deep CFR while PhD at CMU
- Created Libratus and Pluribus, the world's first superhuman poker AIs
- Research scientist currently at OpenAI, previously worked at Meta AI
- Leading role in developing OpenAI's latest GPT o1 and o3 LLMs

Pinned

Noam Brown @polynoamial · Sep 12, 2024

Today, I'm excited to share with you all the fruit of our effort at @OpenAI to create AI models capable of truly general reasoning: OpenAI's new o1 model series! (aka 🧠) Let me explain 📊 1/

Benchmark	Model	Accuracy (%)
Competition Math (AIME 2024)	gpt4o	13.4
	o1 preview	56.7
	o1	83.3
Competition Code (CodeForces)	gpt4o	11.0
	o1 preview	62.0
	o1	89.0
PhD-Level Science Questions (GPQA Diamond)	gpt4o	56.1
	o1 preview	78.3
	o1	78.0
	expert human	69.7

222 1.9K 11K 2.4M



A large, stylized flower graphic composed of several overlapping circles in a lighter shade of blue, positioned on the left side of the image.

Giveaway Time!

Just Your Standard Guessing Game

- Guess a non-negative real number: $[0, \infty)$
- Closest guess (by absolute difference) to the standard deviation of the guesses wins!
 - Ties broken by earliest submission
- Prize: Beats Solo 4
- Quiz is gonna close 1 minute after starting!!!
- Ready?

pkr.bot/jysgg

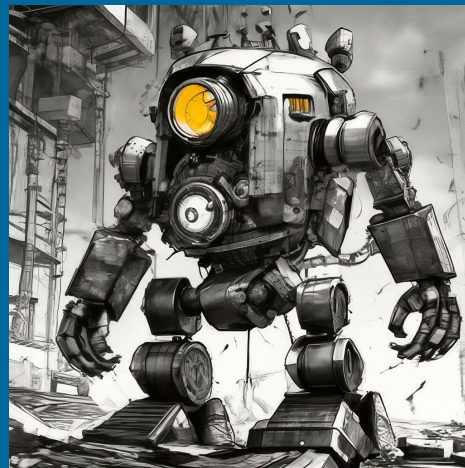
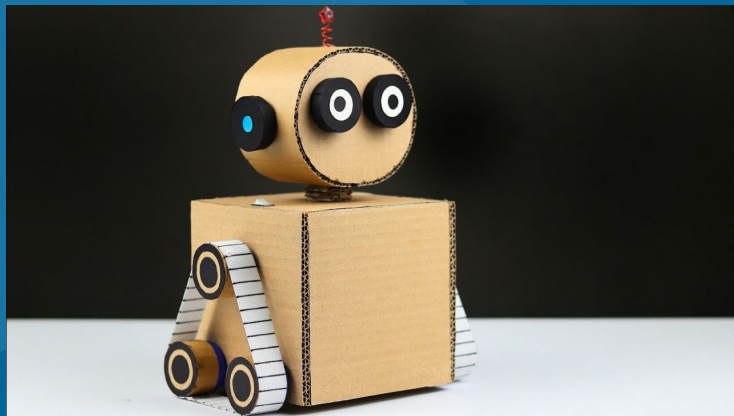


Counterfactual Regret Minimization (CFR)

Paco Gomez-Paz

What is CFR?

- “Self play” algorithm to “solve” zero-sum two player imperfect information games that learns from “regret”



Defining Terms

- Zero Sum: Whatever one player gains, the other player loses
- Imperfect Info: Make decisions without knowing everything.
- Two Player Zero Sum Imperfect Info = Poker Like Games



“Solving” Poker

- **Nash Equilibrium: An unexploitable strategy**
 - No matter how your opponent plays, they can't gain money by adjusting their strategy against you.
- **CFR solves games by finding strategies that are close to a Nash equilibrium**

!! Warning !!

- Pokerbots is a PVP competition
- Find a winning strategy (not an unbeatable one)
- Think about how to exploit (and not get exploited)

LIMPIN' IS
PIMPIN'



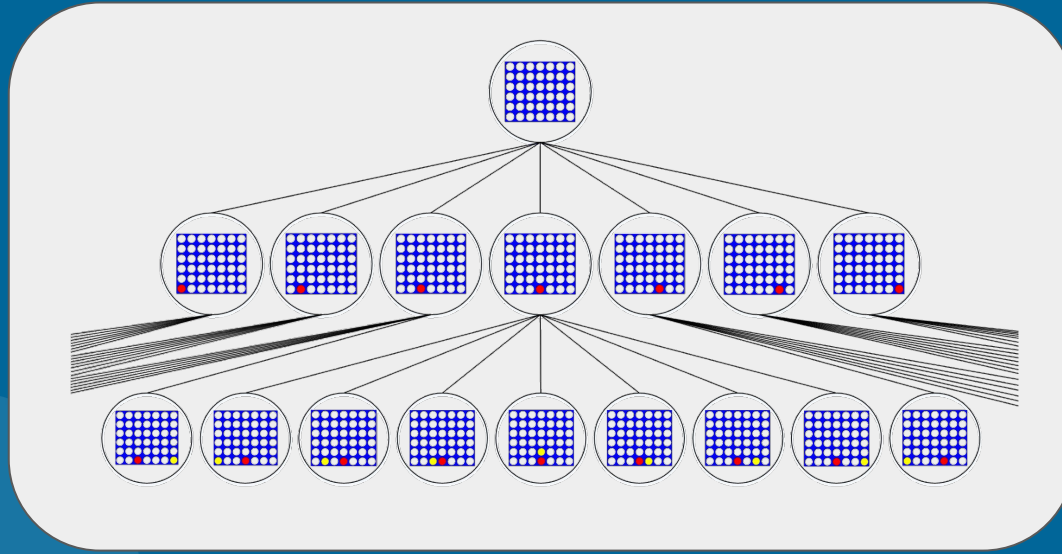
LIMPIN' is PIMPIN'! | How to WIN \$3,000,000

Today

- Build up vocabulary to talk about Poker Like Games
- Formalize “Regret”
- CFR and variants (External Sampling, CFR+, Discounted CFR)
- Practical Concerns and Bucketing



Games as Trees



- Nodes: represent game states
- Edges: Actions/events that transition between game states
- Traversing the Tree = Playing the Game

Nodes

- Decision Node : *Active Player* chooses between available actions
 - Ex : Folding/calling/raising in poker
- Chance Node : Randomness dictates what happens
 - Ex : Whenever a card is dealt
- Terminal Node: Game terminates. Contains player rewards
 - Ex : Showdown

Strategy

- **Strategy:** a probability distribution over actions at each decision node
- **Problem:** Nodes represent specific game states and Parts of a gamestate (eg opponents cards) are not visible to you

Information Sets

- *Information Set (I)* - Groups of game state indistinguishable to a player
 - Determined by visible information: Hole Cards, Board Cards, Stack Sizes, Betting History ...
- **Strategy: probability distribution over actions at each *Information Set***
 - $S[I, A]$ = Probability of Taking Action A at info set I

Evaluating States

- Given an information set I and strategy S
- Assuming both players play S
- Write the expected payoff to the active player as : $E[I | S]$
- “Assuming I was playing against myself how many chips do i expect to win/lose from this spot”

- What is the expected value of this Info Set?



Evaluating Actions

- So far: Assigned value to an Info Set assuming everyone follows S
- Suppose I take an action A , instead of following S , and after that one deviation both players go back to S
- Write the expected value of doing this: $E[I, A \mid S]$

- What's the expected value of this action?



Questions?

Counterfactual Regret and Regret Matching

Counterfactual Regret

- Counterfactual Regret: How much would we regret NOT taking action?
 - how much better or worse I do by taking a specific action instead of playing my usual strategy.
- $\text{Regret}(I, A \mid S) = E(I, A \mid S) - E(I \mid S)$

Higher Regret		Better than existing strategy
Negative Regret		Worse than existing strategy



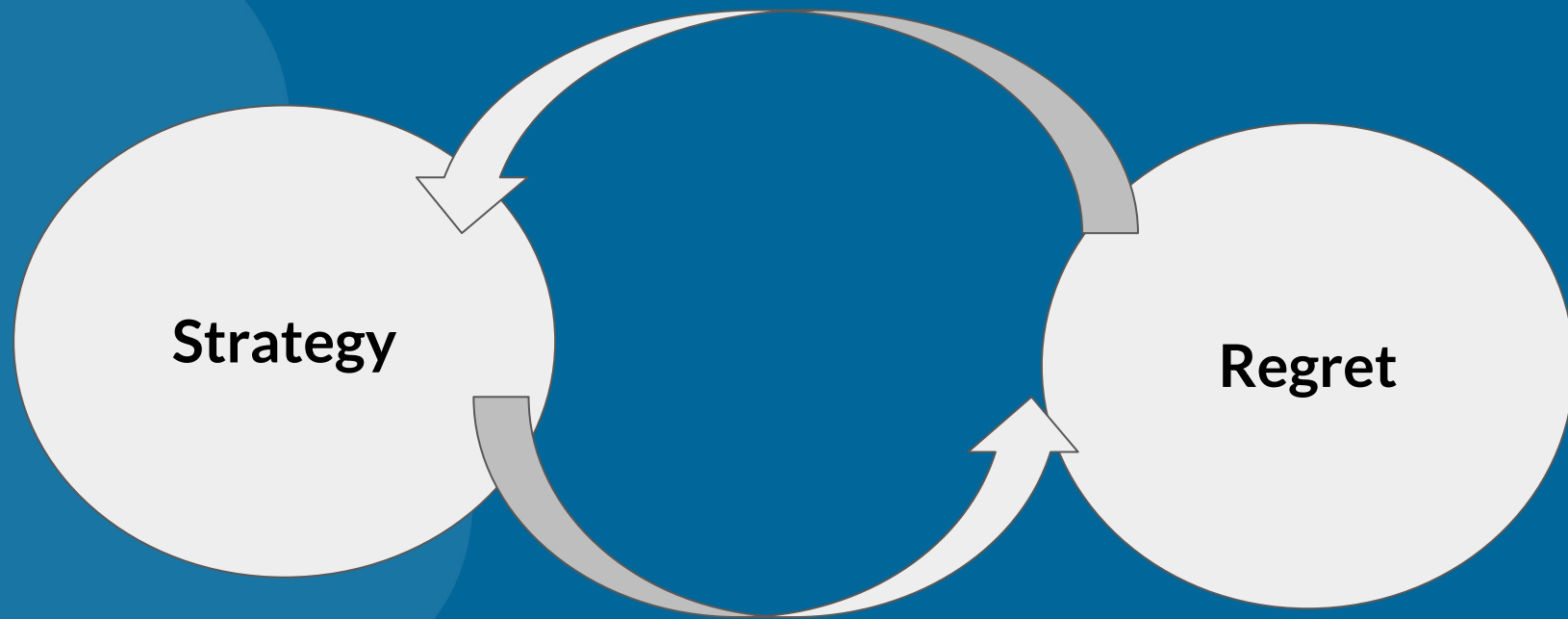
- Counterfactual Regret (All-in with 86o) measures how much better or worse I do by shoving 86o instead of playing my usual strategy.

Regret Matching

- Construct a strategy that learns from a set of regrets
 - 1. Don't play bad actions
 - Given lots of good actions, play better ones more often
- This is called: Regret Matching

$$\text{NewStrategy}[I, A] \propto \max(\text{Regret}[I, A \mid \text{OldStrategy}], 0)$$

$$\text{NewStrategy}[I, A] \propto \max(\text{Regret}[I, A \mid \text{OldStrategy}], 0)$$



$$\text{Regret}[I, A \mid \text{OldStrategy}] = \mathbf{E}[I, A \mid \text{OldStrategy}] - \mathbf{E}[I \mid \text{OldStrategy}]$$

A Self Play Algorithm

- Start with a Strategy and a blank set of regrets
- Calculate (*counterfactual*) regrets for each action at each info set
- Use the regrets to create a new strategy
- Repeat the process above

Strategy^(t)



Strategy^(t+1)

Monte Carlo Sampling for Regret Minimization in Extensive Games

Marc Lanctot

Department of Computing Science
University of Alberta
Edmonton, Alberta, Canada T6G 2E8
lanctot@ualberta.ca

Martin Zinkevich

Yahoo! Research
Santa Clara, CA, USA 95054
maz@yahoo-inc.com

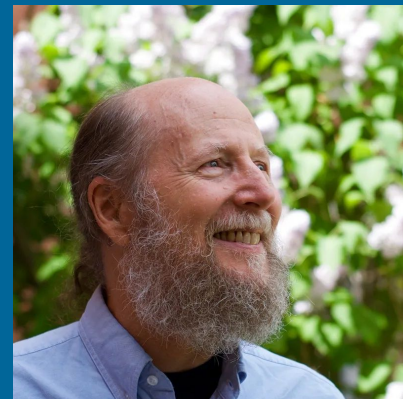
Kevin Waugh

School of Computer Science
Carnegie Mellon University
Pittsburgh PA 15213-3891
waugh@cs.cmu.edu

Michael Bowling

Department of Computing Science
University of Alberta
Edmonton, Alberta, Canada T6G 2E8
bowling@cs.ualberta.ca

Use *cumulative* regrets and return the *average* strategy as your output



Theorem 3 When using vanilla CFR for player i , $R_i^T \leq \Delta_{u,i} M_i \sqrt{|A_i|} / \sqrt{T}$.

Algorithm 1 Vanilla CFR

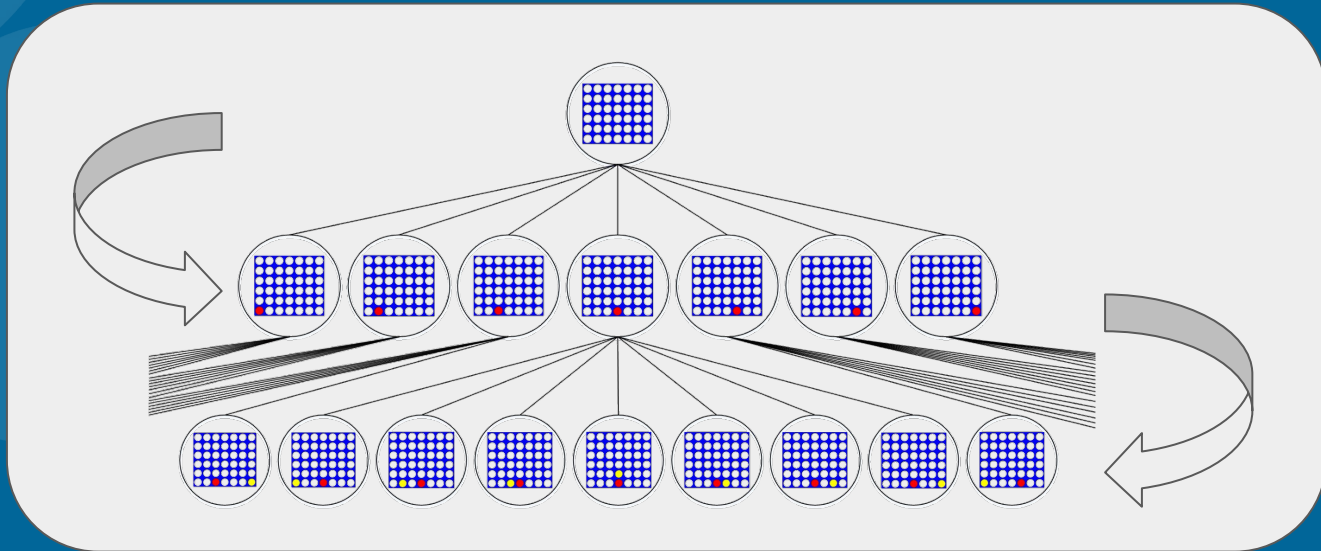
- 1: Initialize cumulative regrets: $\text{CumRegret}[I, A] \leftarrow 0 \quad \forall I, A$
 - 2: Initialize strategy: $\text{Strategy}^{(0)}[I, A] \leftarrow \frac{1}{|\mathcal{A}(I)|} \quad \forall I, A \in \mathcal{A}(I)$
 - 3: **for** $t = 0$ to $T - 1$ **do**
 - 4: Compute regrets under $\text{Strategy}^{(t)}$:
 $\text{Regret}[I, A \mid \text{Strategy}^{(t)}] = \mathbf{E}[I, A \mid \text{Strategy}^{(t)}] - \mathbf{E}[I \mid \text{Strategy}^{(t)}]$
 - 5: Accumulate regrets:
 $\text{CumRegret}[I, A] \leftarrow \text{CumRegret}[I, A] + \text{Regret}[I, A \mid \text{Strategy}^{(t)}]$
 - 6: Update strategy via regret matching:
 $\text{Strategy}^{(t+1)}[I, A] \propto \max(\text{CumRegret}[I, A], 0)$
 - 7: **end for**
 - 8: Compute average strategy:
 $\text{AverageStrategy}[I, A] = \left(\frac{1}{T}\right) \sum_{t=1}^T \text{Strategy}^{(t)}[I, A]$
 - 9: **return** AverageStrategy
-

Questions?

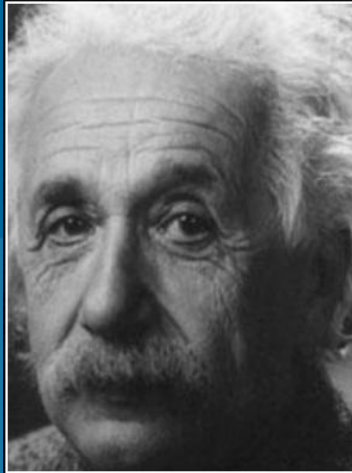
External Sampling

Monte Carlo CFR

$$\mathbf{E}[I \mid S] = \sum_{A \in \text{Legal Actions}} \text{Strategy}[I, A] \mathbf{E}[I, A \mid \text{Strategy}]$$



Theory vs Practice



In theory, theory and practice are
the same. In practice, they are not.

— *Albert Einstein* —

AZ QUOTES

How do we do this calculation on a real world computer?

Sampling with Rollouts

- Don't compute the expectation, estimate by sampling
- Roll Outs give noisy but unbiased estimates

Algorithm 1 SampleRollout(N , CumRegret)

```
1: function SAMPLEROLLOUT( $N$ , CumRegret)
2:   if  $N$  is terminal then
3:     return Rewards( $N$ )
4:   end if
5:   if  $N$  is a chance node then
6:     Sample  $A \sim P(\cdot \mid N)$ 
7:     return SAMPLEROLLOUT( $N(A)$ , CumRegret)
8:   end if
9:    $p \leftarrow P(N)$  ▷ player to act
10:   $I \leftarrow I(N)$ 
11:   $S_p(\cdot) \leftarrow \text{RegretMatching}(\text{CumRegret}[p, I, \cdot])$ 
12:  Sample  $A \sim S_p$ 
13:  return SAMPLEROLLOUT( $N(A)$ , CumRegret)
14: end function
```

Example Update Step

Algorithm 1 UpdateAtDecisionNode(N , CumRegret, AvgStrategy)

```
1: function UPDATEATDECISIONNODE( $N$ , CumRegret, AvgStrategy)
2:    $p \leftarrow P(N)$  ▷ player to act
3:    $I \leftarrow I(N)$  ▷ info set of  $p$  at  $N$ 
4:    $S(\cdot) \leftarrow \text{RegretMatching}(\text{CumRegret}[p, I, \cdot])$ 
5:   for all  $a \in \mathcal{A}(N)$  do
6:      $R[a, \cdot] \leftarrow \text{SAMPLEROLLOUT}(N(a), \text{CumRegret})$ 
7:      $u[a] \leftarrow R[a, p]$  ▷ utility for acting player
8:   end for
9:    $v \leftarrow \sum_{a \in \mathcal{A}(N)} S(a) u[a]$ 
10:  for all  $a \in \mathcal{A}(N)$  do
11:     $\text{CumRegret}[p, I, a] \leftarrow \text{CumRegret}[p, I, a] + (u[a] - v)$ 
12:  end for
13:  for all  $a \in \mathcal{A}(N)$  do
14:     $\text{AvgStrategy}[p, I, a] \leftarrow \text{AvgStrategy}[p, I, a] + S(a)$ 
15:  end for
16: end function
```

Weighting Updates

- We can do better than pure sampling
 - Maintain cumulative probabilities of each players set of actions
 - Weight updates to Cumulative Regret and Average Strategy by the opponents/players reach probability

Algorithm 1 UpdateAtDecisionNode(N , CumProbs, CumRegret, AvgStrategy)

```
1: function UPDATEATDECISIONNODE( $N$ , CumProbs, CumRegret, AvgStrategy)
2:    $p \leftarrow P(N)$  ▷ player to act
3:    $I \leftarrow I(N)$  ▷ info set of  $p$  at  $N$ 
4:    $S(\cdot) \leftarrow \text{RegretMatching}(\text{CumRegret}[p, I, \cdot])$ 
5:    $o \leftarrow \text{Opp}(p)$  ▷ the other player (2-player)
6:   for all  $a \in \mathcal{A}(N)$  do
7:      $R[a, \cdot] \leftarrow \text{SAMPLEROLLOUT}(N(a), \text{CumProbs}, \text{CumRegret})$ 
8:   end for
9:    $v \leftarrow \sum_{a \in \mathcal{A}(N)} S(a) u[a]$ 
   ▷ Weight Regret Update by opponent reach
10:  for all  $a \in \mathcal{A}(N)$  do
11:     $\text{CumRegret}[p, I, a] \leftarrow \text{CumRegret}[p, I, a] + \text{CumProbs}[o] \cdot (u[a] - v)$ 
12:  end for
   ▷ Weight Average strategy update by Player Reach
13:  for all  $a \in \mathcal{A}(N)$  do
14:     $\text{AvgStrategy}[p, I, a] \leftarrow \text{AvgStrategy}[p, I, a] + \text{CumProbs}[p] \cdot S(a)$ 
15:  end for
16: end function
```

External Sampling CFR

- Initialize CumRegret and AverageStrategy at 0
- Use Regret Matching strategy to play poker
- Call UpdateAtDecisionNode during playthroughs to update CumRegret/AverageStrategy
- Note: Sampling the game tree keeps us focused on “important” nodes

Recursion

- We do two traversals per update:
 - Tree Traversal: Follow trajectory to update params
 - Rollout Traversal: Call SampleRollout to estimate action
- Can Collapse Into one Recursive Traversal:
 - Return Sampled reward in the update function
 - Get to do Updates and Value Estimation in same pass

Algorithm 1 External Sampling Monte Carlo CFR (MCCFR)

```

1: Initialize cumulative regrets:  $\text{CumRegret}[I, A] \leftarrow 0 \quad \forall I, A$ 
2: Initialize cumulative strategy:  $\text{CumStrat}[I, A] \leftarrow 0 \quad \forall I, A$ 
3: for  $t = 0$  to  $T - 1$  do
4:   for player  $i \in \{1, \dots, n\}$  do
5:      $\text{ES\_TRAVERSE}(N_0, i, 1, 1)$ 
6:   end for
7: end for
8: Compute average strategy:
9:  $\bar{S}(I, A) \leftarrow \frac{\text{CumStrat}[I, A]}{\sum_{A' \in \mathcal{A}(I)} \text{CumStrat}[I, A']} \quad \forall I, A$ 
10: return  $\bar{S}$ 

```

Algorithm 1 ESTraverse($N, i, \text{CumProbs}, \text{CumRegret}, \text{CumStrat}$) (External Sampling Traverse)

```

1: function ESTRAVERSE( $N, i, \text{CumProbs}, \text{CumRegret}, \text{CumStrat}$ )
2:   if  $N$  is terminal then
3:     return Rewards( $N$ )
4:   end if
5:   if  $N$  is a chance node then
6:     Sample  $A \sim P(\cdot \mid N)$ 
7:     return ESTRAVERSE( $N(A), i, \text{CumProbs}, \text{CumRegret}, \text{CumStrat}$ )
8:   end if
9:    $p \leftarrow P(N)$  ▷ player to act
10:   $I \leftarrow I(N)$  ▷ infoset of  $p$  at  $N$ 
11:  Strategy( $\cdot$ )  $\leftarrow$  RegretMatching( $\text{CumRegret}[p, I, \cdot]$ )
12:  if  $p \neq i$  then ▷ External-sample opponent
13:    Sample  $A \sim \text{Strategy}(\cdot)$ 
14:     $\text{CumProbs}[p] \leftarrow \text{CumProbs}[p] \cdot \text{Strategy}(A)$ 
15:    return ESTRAVERSE( $N(A), i, \text{CumProbs}, \text{CumRegret}, \text{CumStrat}$ )
16:  else ▷ Traverser update (sweep actions)
17:     $o \leftarrow \text{Opp}(p)$ 
18:    for all  $a \in \mathcal{A}(N)$  do
19:       $\text{CumProbsChild} \leftarrow \text{CumProbs}$  ▷ local copy
20:       $\text{CumProbsChild}[p] \leftarrow \text{CumProbsChild}[p] \cdot \text{Strategy}(a)$ 
21:       $R[a, \cdot] \leftarrow \text{ESTRAVERSE}(N(a), i, \text{CumProbsChild}, \text{CumRegret}, \text{CumStrat})$ 
22:       $u[a] \leftarrow R[a, p]$ 
23:    end for
24:     $\text{NodeValue} \leftarrow \sum_{a \in \mathcal{A}(N)} \text{Strategy}(a) u[a]$ 
25:    for all  $a \in \mathcal{A}(N)$  do
26:       $\text{CumRegret}[p, I, a] \leftarrow \text{CumRegret}[p, I, a] + \text{CumProbs}[o] \cdot (u[a] - \text{NodeValue})$ 
27:    end for
28:    for all  $a \in \mathcal{A}(N)$  do
29:       $\text{CumStrat}[p, I, a] \leftarrow \text{CumStrat}[p, I, a] + \text{CumProbs}[p] \cdot \text{Strategy}(a)$ 
30:    end for
31:    RewardsOut[ $p$ ]  $\leftarrow$  NodeValue; RewardsOut[ $o$ ]  $\leftarrow -\text{NodeValue}$ 
32:    return RewardsOut
33:  end if
34: end function

```

Beyond ES MCCFR

- Related Methods: Outcome Sampling CFR, Deep CFR ...
- Practical Speed Ups: CFR+,
- Upweight later strategies in our Averaged Strategy
 - Discounted/Linear CFR
- You should Explore!!

Questions?

Lunch 🤪

Leave any type of feedback at pkr.bot/feedback!

SUBWAY



Bucketing/Abstraction

Bucketing

- Poker is a very Big Game
- Goal: Reduce the number of infosets to give our MCCFR a better chance

Potential-Aware Imperfect-Recall Abstraction with Earth Mover's Distance in Imperfect-Information Games

Sam Ganzfried and Tuomas Sandholm

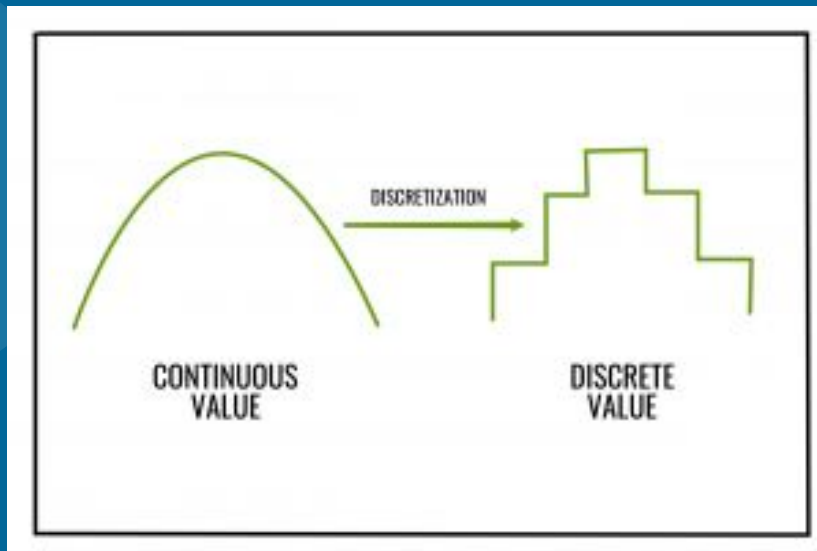
Computer Science Department
Carnegie Mellon University
{sganzfri, sandholm}@cs.cmu.edu

Poker Specifics

- A Poker Node is determined by:
 - Action History
 - Public Cards + Your Hole Cards
 - A Node is determined by:
- Bucketing: Compressing poker down to a simplified game
That still allows us to develop a reasonable strategy

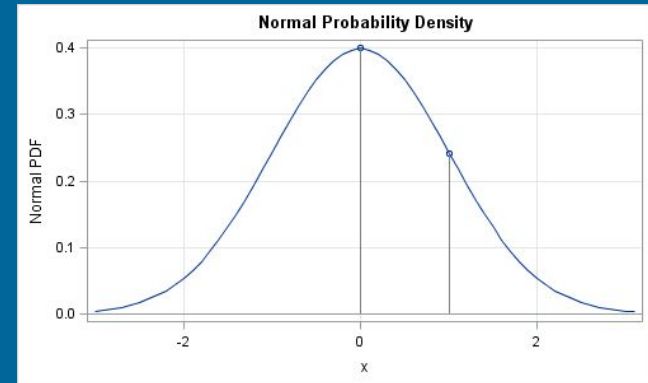
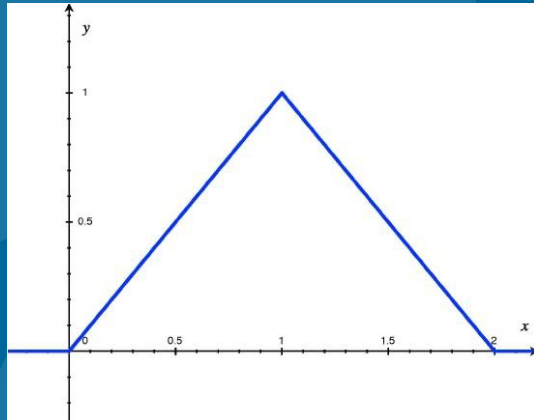
Action Discretization

- Legal Raises: Min Raise, Min Raise + 1 ... Max Raise
- Discretized Raises: Half Pot, Pot, 3/2 Pot ...



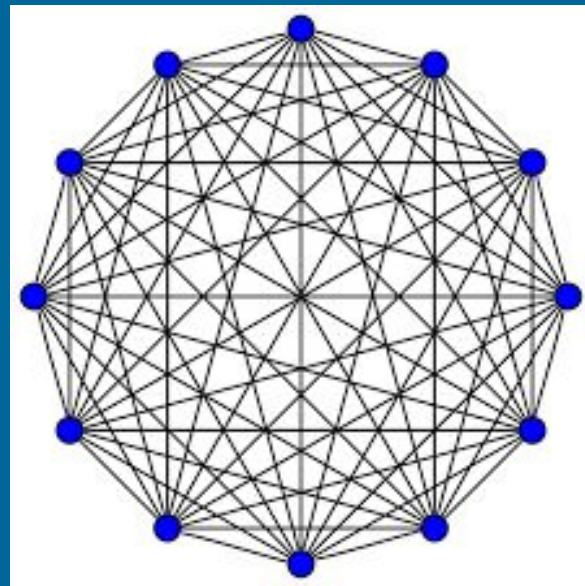
Earth Mover's Distance

- Minimum work to transform one distribution to another
 - $\text{Work} = \text{Mass moved} \times \text{Distance Moved}$
- 1D Distribution: Imagine moving mass in PDF curve



EMD on Graphs

- EMD on Graphs: Moving mass between nodes on a Fully Connected Graph with a distance matrix



“K-Means” Clustering

Algorithm 1 GeneralizedKMeans(X, K, D)

```
1: function GENERALIZEDKMEANS( $X, K, D$ )
2:    $C \leftarrow \text{INITIALIZECENTERS}(X, K)$   $\triangleright C = \{c_1, \dots, c_K\}$ 
3:   repeat  $\triangleright$  Label points by nearest center under  $D$ 
4:     for  $i = 1$  to  $n$  do
5:        $\text{Label}[i] \leftarrow \arg \min_{k \in \{1, \dots, K\}} D(x_i, c_k)$ 
6:     end for
7:      $\triangleright$  Update centers to minimize within-cluster distance
8:     for  $k = 1$  to  $K$  do
9:        $C_k \leftarrow \{x_i : \text{Label}[i] = k\}$ 
10:       $c_k \leftarrow \arg \min_{c \in C} \sum_{x \in C_k} D(x, c)$ 
11:    end for
12:  until converged
13:  return  $C, \text{Label}$ 
14: end function
```

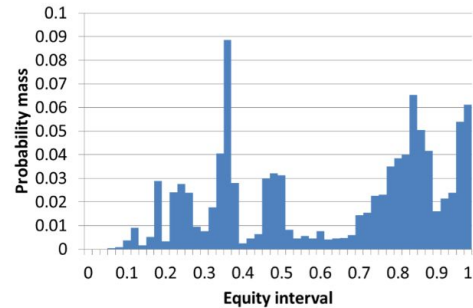
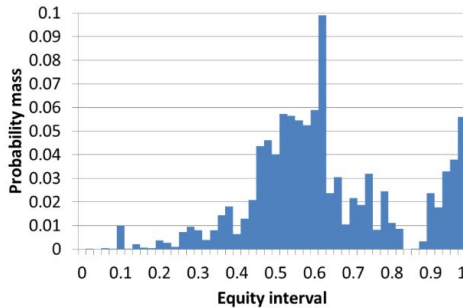
Bucketing: River

- Cluster Hands by strength
- First Order Approx: Percent of random hands you beat
- Run L2 norm clustering on rivers by “Strength”



Bucketing : Turn

- 46 possible river cards, each producing a different equity
- Use outcomes to build Equity Histogram for each turn
- Cluster turn states using EMD on the histograms



Flop and Preflop

- **Similar Idea:**
 - Can look at the distribution over next street buckets
 - Use outcomes to build Equity Histogram for each turn
- **How do we run EMD on this?**
 - Measure distance between next streets labels as the distance between their centers

Problems

- EMD on graphs is expensive (greedy approximations)
- Memory is limited:
 - Can't store labels for every possible street
 - Canonicalization + Online label assignment
- Unseen Opponent Actions:
 - Opponent plays outside our discretized action set
 - Ex from Last Year: Min Bet on large pot

Some History

- Dr. Noam Brown: co-created Deep CFR while PhD student at CMU
 - Completed in Pokerbots while CMU grad student
 - Created Pluribus (Poker) and Cicero (Diplomacy) at FAIR
 - Worked on reasoning in LLMs at OpenAI (o3 and o1)
 - Is giving a lecture on Wednesday, January 28th!

Deep Counterfactual Regret Minimization

Noam Brown*

Computer Science Department
Carnegie Mellon University
Pittsburgh, PA 15213
noamb@cs.cmu.edu

Adam Lerer*

Facebook AI Research
New York, NY 10003
alerer@fb.com

Sam Gross

Facebook AI Research
New York, NY 10003
sgross@fb.com

Tuomas Sandholm

Computer Science Department
Carnegie Mellon University
Pittsburgh, PA 15213
sandholm@cs.cmu.edu

DIPLOMACY GAMEPLAY

Playing Diplomacy with CICERO

Andrew Goff

Diplomacy World Champion



Noam Brown

Meta AI Research Scientist



Adam Lerer

Meta AI Research Engineer



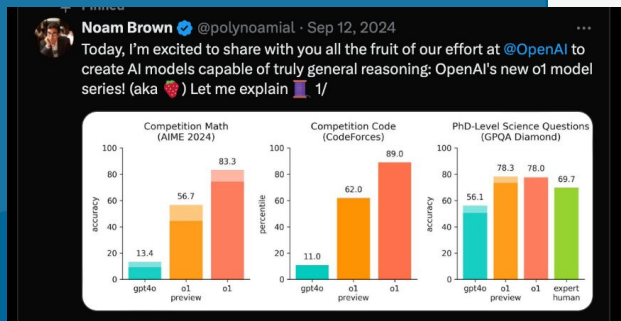
CICERO

AI Agent

Superhuman AI for multiplayer poker

NOAM BROWN AND TUOMAS SANDHOLM [Authors Info & Affiliations](#)

SCIENCE 33, 1413019 (2019) | Vol 365, Issue 6455 | pp. 885-890 | DOI:10.1126/science.1240997



The image features a solid blue background. On the left side, there are several overlapping circles of varying shades of blue, creating a decorative, floral-like pattern. The text "Giveaway Winner" is centered horizontally and vertically in a white, serif font.

Giveaway Winner

Just Your Standard Guessing Game

- A little analysis:
 - Standard deviation ~ average distance from mean
 - Numbers all over the place (guesses ranged from 0 to 10^{259})
 - Mean = sum / count, but sum is basically ~ max here, so
 - Mean ~ max / count
 - Std. dev = Mean of: [abs(mean - e_1), abs(mean - e_2), ...]
 - ~ mean of [mean, mean, ..., mean, max - mean]
 - ~ Max / (count/2)
 - So if max is 10^{259} , and no guesses are near it, std. dev will be $\sim 8 * 10^{257}$!

Just Your Standard Guessing Game

WINNER:

lando

Guess: 10^{182}



Thanks for watching!

Information hub at pkr.bot/hub

Slides/notes will be posted on pkr.bot/resources

Make sure to check pkr.bot/piazza for updates

Lecture recordings at pkr.bot/panopto

Leave feedback at pkr.bot/feedback!